

OOP in C++: Constructors, Destructors, Copy/Move, Virtual Functions, vtable/vptr

1. Overview

Object-Oriented Programming in C++ is a way of designing programs using objects that combine data and behavior. In interviews, this topic usually focuses on how objects are created, copied, moved, destroyed, and how runtime polymorphism works.

Definition

In C++, OOP uses classes and objects to model real-world or system-level entities. The key mechanisms in this guide are:

- **Constructor:** Initializes an object when it is created.
- **Destructor:** Cleans up when an object is destroyed.
- **Copy constructor:** Creates a new object as a copy of another object.
- **Copy assignment operator:** Copies data into an already existing object.
- **Move constructor:** Transfers resources from a temporary object to a new object.
- **Virtual function:** Enables runtime polymorphism.
- **Pure virtual function:** Makes a class abstract.
- **vtable/vptr:** Internal mechanism used by most C++ compilers to implement virtual functions.

Why It Matters

C++ gives you direct control over object lifetime and resources such as heap memory, file handles, sockets, locks, and database connections. If you do not understand constructors, destructors, copying, moving, and virtual dispatch, you can easily create memory leaks, double deletes, slicing bugs, slow copies, or incorrect polymorphic behavior.

Where It Is Used in Real Systems

- **Operating systems:** RAIL wrappers for locks, file descriptors, and kernel handles.
- **Databases:** Buffer managers, query operators, transaction objects.
- **Browsers:** DOM nodes, rendering objects, event handlers.
- **Backend servers:** Request objects, socket wrappers, connection pools.
- **Game engines:** Entity components, resource managers, polymorphic behavior trees.
- **Application code:** Smart pointers, containers, inheritance hierarchies, plugin systems.

Why Interviewers Ask About It

Interviewers ask this topic because it tests whether you understand:

- Object lifecycle in C++.
- Resource management.
- Difference between initialization and assignment.
- Deep copy vs shallow copy.

- Runtime polymorphism.
- Why destructors should often be virtual in base classes.
- Performance benefits of move semantics.
- Internal memory model behind virtual functions.

2. Core Idea

The core idea is simple: a C++ object has a lifecycle.

Object lifecycle:

```
Memory allocated
  |
Constructor runs
  |
Object is used
  |
Object may be copied, assigned, or moved
  |
Destructor runs
  |
Memory/resources released
```

Intuition

Think of an object as a rented apartment.

- Constructor: You move in and set up furniture.
- Destructor: You clean up and return the keys.
- Copy constructor: Someone creates a new apartment with the same setup.
- Assignment operator: An existing apartment changes its setup to match another.
- Move constructor: Instead of copying furniture, you transfer ownership of the furniture.
- Virtual function: You call a common action, but the actual behavior depends on the real object type.
- Pure virtual function: A rule saying every concrete child class must define this behavior.

Small Example

```
#include <iostream>
#include <string>

class File {
    std::string name;

public:
    File(const std::string& fileName) : name(fileName) {
        std::cout << "Opening file: " << name << '\n';
    }

    ~File() {
```

```
        std::cout << "Closing file: " << name << '\n';
    }
};

int main() {
    File f("data.txt");
}
```

Step-by-Step Explanation

1. `File f("data.txt");` creates an object.
2. The constructor runs and initializes `name`.
3. The object is used inside `main`.
4. When `main` ends, `f` goes out of scope.
5. The destructor runs automatically.

This automatic cleanup pattern is called **RAII**: Resource Acquisition Is Initialization. It is one of the most important idioms in C++.

3. Important Subtopics

3.1 Constructors

What It Means

A constructor is a special member function that runs automatically when an object is created. It initializes the object.

```
class Student {
    int rollNo;
    std::string name;

public:
    Student(int r, std::string n) : rollNo(r), name(n) {}
};
```

Why It Matters

Constructors ensure objects start in a valid state. Without proper initialization, objects may contain garbage values or invalid resources.

Example

```
Student s(101, "Asha");
```

Here, the constructor initializes `rollNo` and `name`.

Common Interview Angle

Interviewers may ask:

- Difference between default, parameterized, and copy constructors.
- Why initializer lists are preferred.
- Constructor call order in inheritance.

3.2 Destructor

What It Means

A destructor is called automatically when an object is destroyed.

```
class Buffer {  
    int* data;  
  
public:  
    Buffer(int size) {  
        data = new int[size];  
    }  
  
    ~Buffer() {  
        delete[] data;  
    }  
};
```

Why It Matters

Destructors prevent resource leaks. They are used to release heap memory, files, sockets, locks, and other resources.

Example

```
{  
    Buffer b(100);  
} // destructor runs here
```

Common Interview Angle

Interviewers often ask:

- When is a destructor called?
- Why should a base class destructor be virtual?
- Can a destructor be overloaded?
- What happens if a destructor throws?

3.3 Copy Constructor

What It Means

A copy constructor creates a new object from an existing object.

```
class Box {  
    int value;  
  
public:  
    Box(int v) : value(v) {}  
  
    Box(const Box& other) : value(other.value) {}  
};
```

Why It Matters

If a class owns a resource, the default copy constructor may perform a shallow copy, causing bugs.

Example

```
Box b1(10);  
Box b2 = b1; // copy constructor
```

Common Interview Angle

Interviewers ask:

- When is the copy constructor called?
- Why should the parameter be `const ClassName&`?
- What is shallow copy vs deep copy?
- How does copy constructor differ from assignment operator?

3.4 Copy Assignment Operator

What It Means

The copy assignment operator copies one object into another already existing object.

```
class Box {  
    int value;  
  
public:  
    Box(int v = 0) : value(v) {}  
  
    Box& operator=(const Box& other) {
```

```
        if (this != &other) {
            value = other.value;
        }
        return *this;
    }
};
```

Why It Matters

Assignment must correctly handle existing resources and self-assignment.

Example

```
Box b1(10);
Box b2(20);
b2 = b1; // copy assignment operator
```

Common Interview Angle

Interviewers may ask:

- Why return `Box&`?
- Why check `this != &other`?
- Difference between initialization and assignment.

3.5 Move Constructor

What It Means

A move constructor creates a new object by taking resources from a temporary or expiring object.

```
class Buffer {
    int* data;

public:
    Buffer(int size) : data(new int[size]) {}

    ~Buffer() {
        delete[] data;
    }

    Buffer(Buffer&& other) noexcept : data(other.data) {
        other.data = nullptr;
    }
};
```

Why It Matters

Moving avoids expensive deep copies. It is important for performance in modern C++.

Example

```
Buffer makeBuffer() {  
    return Buffer(1000);  
}  
  
Buffer b = makeBuffer(); // move may be used, or copy elision may occur
```

Common Interview Angle

Interviewers ask:

- What is an rvalue reference?
- Why use `noexcept` in move constructors?
- What state should the moved-from object be in?
- Difference between copy and move.

3.6 Virtual Functions

What It Means

A virtual function allows a function call through a base class pointer/reference to execute the derived class version at runtime.

```
class Shape {  
public:  
    virtual void draw() {  
        std::cout << "Drawing shape\n";  
    }  
};  
  
class Circle : public Shape {  
public:  
    void draw() override {  
        std::cout << "Drawing circle\n";  
    }  
};
```

Why It Matters

Virtual functions enable runtime polymorphism.

Example

```
Shape* s = new Circle();  
s->draw(); // Drawing circle  
delete s;
```

Common Interview Angle

Interviewers ask:

- What is runtime polymorphism?
- How is virtual dispatch implemented?
- Can constructors be virtual?
- Why should destructors be virtual in polymorphic base classes?

3.7 Pure Virtual Functions

What It Means

A pure virtual function has no required implementation in the base class and makes the class abstract.

```
class Shape {  
public:  
    virtual double area() = 0;  
};
```

Why It Matters

Pure virtual functions define an interface. They force derived classes to provide behavior.

Example

```
class Circle : public Shape {  
    double radius;  
  
public:  
    Circle(double r) : radius(r) {}  
  
    double area() override {  
        return 3.14159 * radius * radius;  
    }  
};
```

Common Interview Angle

Interviewers ask:

- What is an abstract class?
- Can an abstract class have constructors?
- Can pure virtual functions have definitions?
- Difference between interface and abstract class in C++.

3.8 vtable and vptr

What It Means

Most C++ compilers implement virtual functions using:

- **vtable**: A table of function pointers for virtual functions.
- **vptr**: A hidden pointer inside each polymorphic object pointing to its class's vtable.

Why It Matters

This explains how runtime polymorphism works and why virtual functions have a small overhead.

Example

```
class Base {
public:
    virtual void show() {}
};

class Derived : public Base {
public:
    void show() override {}
};
```

Conceptually:

```
Derived object
+-----+
| vptr -----|----> Derived vtable
| data members |      |
+-----+          +---> Derived::show()
```

Common Interview Angle

Interviewers ask:

- What is vtable?
- What is vptr?
- Is vtable per object or per class?
- What happens during virtual function call?
- Does every class have a vtable?

4. Real-World Example

Backend Server Request Handlers

Suppose a backend server supports different request handlers:

```
class RequestHandler {
public:
    virtual void handle() = 0;
    virtual ~RequestHandler() = default;
};

class LoginHandler : public RequestHandler {
public:
    void handle() override {
        std::cout << "Handling login\n";
    }
};

class PaymentHandler : public RequestHandler {
public:
    void handle() override {
        std::cout << "Handling payment\n";
    }
};
```

Usage:

```
void process(RequestHandler& handler) {
    handler.handle();
}
```

Why this is useful:

- The server can treat all handlers uniformly.
- New handlers can be added without changing core server logic.
- Virtual functions allow the correct handler behavior at runtime.
- Virtual destructor ensures correct cleanup through base pointers.

5. Diagrams / Mental Models

Object Lifecycle

```
Creation
|
v
Constructor
```

```

    |
    v
Valid object
    |
    +--> Copy constructor: creates a new copy
    |
    +--> Assignment operator: replaces existing object's state
    |
    +--> Move constructor: transfers resources to new object
    |
    v
Destructor
    |
    v
Resources released

```

Copy vs Assignment

Copy constructor:

```
Box b1(10);
Box b2 = b1;
```

Creates b2 from b1.

Assignment operator:

```
Box b1(10);
Box b2(20);
b2 = b1;
```

b2 already exists, then receives b1's value.

Virtual Function Dispatch

```
Base* ptr = new Derived();
ptr->show();
```

Step 1: Compiler sees show() is virtual.

Step 2: Runtime object has vptr.

Step 3: vptr points to Derived vtable.

Step 4: Function pointer for show() resolves to Derived::show().

Step 5: Derived::show() executes.

vtable/vptr Mental Model

```

Class Base vtable
+-----+
| Base::show      |
| Base::print     |
+-----+

Class Derived vtable
+-----+
| Derived::show   |
| Base::print     |
+-----+

Derived object
+-----+
| hidden vptr ----|----> Derived vtable
| member variables |
+-----+

```

6. Common Interview Questions

1. What is a constructor in C++?

A constructor is a special member function that runs automatically when an object is created. It initializes the object.

Key points interviewer expects:

- Same name as class.
- No return type.
- Can be overloaded.
- Used for initialization.
- Prefer initializer lists for member initialization.

Common mistakes:

- Saying constructors return `void`.
- Confusing constructor with normal member function.
- Ignoring initializer lists.

2. What is a destructor?

A destructor is a special member function that runs automatically when an object is destroyed. It is used to release resources.

Key points interviewer expects:

- Name is `~ClassName`.
- No parameters.
- Cannot be overloaded.
- Called automatically when object lifetime ends.

- Should be virtual in polymorphic base classes.

Common mistakes:

- Saying destructors must be called manually.
- Forgetting that local objects are destroyed automatically.
- Not mentioning virtual destructors.

3. What is the difference between copy constructor and assignment operator?

The copy constructor creates a new object from an existing object. The assignment operator copies into an object that already exists.

```
Student s2 = s1; // copy constructor  
s2 = s1;        // assignment operator
```

Key points interviewer expects:

- Copy constructor initializes a new object.
- Assignment modifies an existing object.
- Assignment should handle self-assignment.

Common mistakes:

- Treating both as exactly the same.
- Not knowing when each one is called.

4. What is shallow copy vs deep copy?

A shallow copy copies pointer values, so two objects may point to the same memory. A deep copy creates separate memory and copies the actual data.

Key points interviewer expects:

- Shallow copy can cause double delete.
- Deep copy is required for owning raw pointers.
- Smart pointers reduce the need for manual copy logic.

Common mistakes:

- Saying shallow copy is always wrong.
- Forgetting ownership context.

5. What is the Rule of Three?

If a class needs a custom destructor, copy constructor, or copy assignment operator, it probably needs all three.

Key points interviewer expects:

- Applies to resource-owning classes.

- Prevents leaks, double deletion, and incorrect copying.
- Modern C++ extends this to Rule of Five.

Common mistakes:

- Memorizing the rule without explaining why.
- Not connecting it to resource ownership.

6. What is the Rule of Five?

If a class manages resources, it may need:

- Destructor
- Copy constructor
- Copy assignment operator
- Move constructor
- Move assignment operator

Key points interviewer expects:

- Move operations improve performance.
- Relevant in modern C++.
- Prefer Rule of Zero when possible.

Common mistakes:

- Forgetting move assignment.
- Writing unnecessary special member functions for classes using standard library types.

7. What is a move constructor?

A move constructor creates a new object by taking resources from another object, usually a temporary.

```
Buffer(Buffer&& other) noexcept;
```

Key points interviewer expects:

- Takes an rvalue reference.
- Transfers ownership.
- Leaves moved-from object valid but unspecified.
- Often marked `noexcept`.

Common mistakes:

- Saying move always copies data.
- Using the moved-from object as if unchanged.

8. What is a virtual function?

A virtual function is a member function whose call is resolved at runtime based on the actual object type.

Key points interviewer expects:

- Enables runtime polymorphism.
- Requires base pointer/reference for polymorphic behavior.
- Usually implemented using vtable/vptr.
- Use `override` in derived classes.

Common mistakes:

- Confusing function overloading with overriding.
- Thinking virtual functions are resolved at compile time.

9. What is a pure virtual function?

A pure virtual function is declared using `= 0` and makes a class abstract.

```
virtual void draw() = 0;
```

Key points interviewer expects:

- Derived concrete classes must override it.
- The class cannot be directly instantiated.
- It is used to define interfaces.

Common mistakes:

- Saying pure virtual functions can never have definitions.
- Saying abstract classes cannot have constructors.

10. Why should a base class destructor be virtual?

If a derived object is deleted through a base class pointer, the base destructor must be virtual to ensure the derived destructor also runs.

```
Base* p = new Derived();  
delete p; // safe only if Base has virtual destructor
```

Key points interviewer expects:

- Prevents incomplete destruction.
- Important for polymorphic base classes.
- Avoids resource leaks.

Common mistakes:

- Saying every destructor in every class must be virtual.
- Not linking it to deletion through base pointer.

11. What are vtable and vptr?

The vtable is a table of virtual function addresses. The vptr is a hidden pointer inside a polymorphic object that points to the appropriate vtable.

Key points interviewer expects:

- vtable is usually per class.
- vptr is usually per object.
- Used for runtime virtual dispatch.
- This is implementation detail, not strictly mandated by the C++ standard.

Common mistakes:

- Saying all classes have vtables.
- Saying vtable is stored separately for every object.

12. Can constructors be virtual?

No, constructors cannot be virtual in C++.

Reason:

- During construction, the object is not yet fully formed.
- The dynamic type is being built step by step.
- Virtual dispatch does not work as expected inside constructors.

Key points interviewer expects:

- Constructors cannot be virtual.
- Destructors can and often should be virtual.
- Factory methods can be used when virtual-like creation is needed.

Common mistakes:

- Saying constructors should be virtual for polymorphism.
- Not knowing factory pattern alternative.

7. Deep-Dive Questions

1. What happens if a virtual function is called inside a constructor?

During base class construction, virtual calls do not dispatch to the derived class implementation. They call the version belonging to the class currently being constructed.

```
class Base {  
public:  
    Base() {  
        show();  
    }  
  
    virtual void show() {
```

```

        std::cout << "Base\n";
    }
};

class Derived : public Base {
public:
    void show() override {
        std::cout << "Derived\n";
    }
};

```

Creating `Derived d`; prints `Base`, not `Derived`.

2. Why should move constructors often be marked `noexcept`?

Standard containers like `std::vector` prefer moving elements during reallocation only when move operations are `noexcept`. If moving may throw, the container may copy instead to maintain exception safety.

Important point:

```

MyClass(MyClass&& other) noexcept;

```

This can improve performance.

3. What is object slicing?

Object slicing happens when a derived object is copied into a base object by value. The derived-specific part is lost.

```

class Base {};
class Derived : public Base {
    int extra;
};

Derived d;
Base b = d; // slicing

```

Avoid slicing by using references, pointers, or smart pointers:

```

Base& ref = d;
Base* ptr = &d;

```

4. Can a pure virtual function have a body?

Yes. A pure virtual function can have a definition, but the class remains abstract.

```
class Base {
public:
    virtual void show() = 0;
};

void Base::show() {
    std::cout << "Base implementation\n";
}
```

Derived classes still need to override it if they are to be concrete.

5. How does virtual inheritance affect object layout?

Virtual inheritance is used to solve the diamond problem. It ensures only one shared base subobject exists.

```
class A {};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {};
```

Without virtual inheritance, **D** would contain two **A** subobjects. With virtual inheritance, it contains one shared **A** subobject. Compilers may add hidden pointers or tables to manage this layout.

8. Comparison Tables

Constructor vs Destructor

Feature	Constructor	Destructor
Purpose	Initialize object	Clean up object
Name	Same as class	<code>~ClassName</code>
Return type	None	None
Parameters	Can have parameters	Cannot have parameters
Overloading	Can be overloaded	Cannot be overloaded
Called when	Object is created	Object is destroyed
Interview focus	Initialization, order, initializer list	Cleanup, virtual destructor, RAII

Copy Constructor vs Copy Assignment Operator

Feature	Copy Constructor	Copy Assignment Operator
Purpose	Creates new object from existing object	Copies into existing object

Feature	Copy Constructor	Copy Assignment Operator
Syntax	<code>ClassName(const ClassName& other)</code>	<code>ClassName& operator=(const ClassName& other)</code>
Example	<code>A b = a;</code>	<code>b = a;</code>
Object already exists?	No	Yes
Self-assignment concern	Usually no	Yes
Return value	No return	Usually returns <code>*this</code>

Copy vs Move

Feature	Copy	Move
Meaning	Duplicate resource/value	Transfer resource ownership
Parameter	<code>const T&</code>	<code>T&&</code>
Cost	Can be expensive	Usually cheap
Source object	Unchanged	Valid but unspecified
Used for	Lvalues, normal duplication	Temporaries, expiring objects
Example	<code>T b = a;</code>	<code>T b = std::move(a);</code>

Virtual Function vs Pure Virtual Function

Feature	Virtual Function	Pure Virtual Function
Syntax	<code>virtual void f();</code>	<code>virtual void f() = 0;</code>
Base implementation	Usually exists	May or may not exist
Makes class abstract?	No	Yes
Derived override	Optional	Required for concrete class
Used for	Runtime polymorphism with default behavior	Interface-like behavior

vtable vs vptr

Feature	vtable	vptr
Meaning	Table of virtual function addresses	Hidden pointer to vtable
Usually stored	Per class	Per object
Created for	Classes with virtual functions	Objects of polymorphic classes
Purpose	Stores dispatch targets	Finds correct vtable at runtime

Feature	vtable	vpitr
Standard-mandated?	No, implementation detail	No, implementation detail

Rule of Three vs Rule of Five vs Rule of Zero

Rule	Meaning	Best Used When
Rule of Three	Define destructor, copy constructor, copy assignment together	Old-style resource management
Rule of Five	Add move constructor and move assignment	Manual resource ownership in modern C++
Rule of Zero	Define none of them; use standard library types	Preferred modern C++ style

9. Common Mistakes

- Thinking constructors have a return type.
- Confusing copy constructor with assignment operator.
- Forgetting self-assignment in copy assignment.
- Doing shallow copy for owning raw pointers.
- Forgetting to set moved-from pointer to `nullptr`.
- Using `delete` instead of `delete[]` for arrays.
- Not making base class destructor virtual in polymorphic classes.
- Calling virtual functions in constructors and expecting derived behavior.
- Passing polymorphic objects by value and causing slicing.
- Forgetting `override` in derived classes.
- Thinking every class has a vtable.
- Thinking vtable/vpitr are directly part of the C++ standard.
- Overusing inheritance when composition would be simpler.
- Writing custom copy/move logic when `std::vector`, `std::string`, or smart pointers already handle it.

10. Edge Cases / Special Cases

Constructor Order in Inheritance

Base class constructors run before derived class constructors.

```
Base constructor
Derived constructor
```

Destruction happens in reverse order:

Derived destructor
Base destructor

Member Initialization Order

Members are initialized in the order they are declared in the class, not the order in the initializer list.

```
class Demo {  
    int a;  
    int b;  
  
public:  
    Demo() : b(2), a(1) {}  
};
```

Here, `a` is initialized before `b`.

Virtual Destructor

If a class has virtual functions and is meant to be used polymorphically, give it a virtual destructor.

```
class Base {  
public:  
    virtual ~Base() = default;  
};
```

Pure Virtual Destructor

A destructor can be pure virtual, but it must still have a definition.

```
class Base {  
public:  
    virtual ~Base() = 0;  
};  
  
Base::~~Base() {}
```

Copy Elision

Modern C++ often avoids copy/move entirely through copy elision.

```
std::string makeName() {  
    return std::string("Asha");  
}
```

```
}
```

The returned object may be constructed directly in the caller's storage.

`std::move` Does Not Move by Itself

`std::move` only casts an object to an rvalue reference. The actual move happens when a move constructor or move assignment operator uses it.

```
std::string a = "hello";
std::string b = std::move(a);
```

Moved-From Object

A moved-from object is valid but its value is unspecified. You can destroy it or assign a new value to it.

Virtual Functions and Default Arguments

Default arguments are resolved statically, while virtual functions are dispatched dynamically.

```
class Base {
public:
    virtual void show(int x = 1) {
        std::cout << x << '\n';
    }
};

class Derived : public Base {
public:
    void show(int x = 2) override {
        std::cout << x << '\n';
    }
};

Base* p = new Derived();
p->show(); // uses Base default argument: 1
delete p;
```

11. How to Explain in Interview

C++ OOP is mainly about object lifetime and polymorphism. Constructors initialize objects, destructors clean them up, copy constructor creates a new object from another, assignment operator copies into an existing object, and move constructor transfers resources efficiently from temporary objects. Virtual functions allow runtime polymorphism, pure virtual functions define abstract interfaces, and compilers usually implement this using a vptr inside objects pointing to a vtable of virtual function addresses.

12. Quick Revision Notes

Key Definitions

- Constructor: Initializes an object.
- Destructor: Cleans up an object.
- Copy constructor: Creates a new object from an existing one.
- Copy assignment: Copies into an already existing object.
- Move constructor: Transfers resources from an rvalue.
- Virtual function: Enables runtime dispatch.
- Pure virtual function: Makes class abstract.
- vtable: Table of virtual function addresses.
- vptr: Hidden pointer to vtable.

Important Points

- Use initializer lists for constructors.
- Use virtual destructors in polymorphic base classes.
- Copy constructor is for initialization.
- Assignment operator is for already existing objects.
- Move operations improve performance.
- `std::move` is a cast, not an actual move.
- vtable/vptr are implementation details.
- Avoid object slicing by using references or pointers.

Common Comparisons

- Constructor vs destructor.
- Copy constructor vs assignment operator.
- Copy vs move.
- Virtual vs pure virtual.
- vtable vs vptr.
- Rule of Three vs Rule of Five vs Rule of Zero.

Must-Remember Facts

- Constructors cannot be virtual.
- Destructors can be virtual.
- Destructors cannot be overloaded.
- Pure virtual functions can have definitions.
- Abstract classes can have constructors.
- Base constructors run before derived constructors.
- Derived destructors run before base destructors.
- vtable is usually per class; vptr is usually per object.

Interview Traps

- `Base b = derivedObj;` causes slicing.
- `delete basePtr;` without virtual destructor can be unsafe.
- Virtual calls inside constructors do not call derived overrides.
- Member initialization order follows declaration order.

- Moved-from objects are valid but unspecified.

13. Practice Tasks

Task 1: Trace Constructor and Destructor Order

Write classes `A`, `B`, and `C`, where `C` inherits from `B`, and `B` inherits from `A`. Print messages from each constructor and destructor. Predict the output before running.

Task 2: Implement a Deep Copy Class

Create a class `IntArray` that owns a dynamic array.

Implement:

- Constructor
- Destructor
- Copy constructor
- Copy assignment operator

Check that copying does not share the same memory.

Task 3: Add Move Semantics

Extend `IntArray` with:

- Move constructor
- Move assignment operator

Print logs to see when copy or move happens.

Task 4: Demonstrate Object Slicing

Create a `Base` class and `Derived` class. Assign a `Derived` object to a `Base` variable by value and observe that derived-specific data is lost.

Task 5: Build a Shape Hierarchy

Create an abstract class `Shape` with:

```
virtual double area() = 0;
virtual ~Shape() = default;
```

Implement:

- `Circle`
- `Rectangle`
- `Triangle`

Store them using `std::vector<std::unique_ptr<Shape>>` and calculate total area.

Task 6: Observe Virtual Destructor Behavior

Create a base class without virtual destructor and a derived class that allocates memory. Delete derived object through base pointer. Then make the base destructor virtual and compare behavior.

Task 7: Explain vtable/vptr on Paper

Draw object layout for:

```
class Animal {
public:
    virtual void speak();
};

class Dog : public Animal {
public:
    void speak() override;
};
```

Explain how `Animal* a = new Dog(); a->speak();` resolves to `Dog::speak()`.

14. Final Cheat Sheet

Core Definition

C++ OOP combines object lifecycle management with polymorphism. Constructors create valid objects, destructors clean them up, copy/move operations control how objects are duplicated or transferred, and virtual functions allow runtime behavior selection.

Why It Matters

This topic matters because C++ programs often manage real resources. Correct object lifecycle handling prevents leaks, double deletion, slicing, and performance issues.

Most Asked Questions

- What is the difference between constructor and destructor?
- What is the difference between copy constructor and assignment operator?
- What is shallow copy vs deep copy?
- What is the Rule of Three/Five/Zero?
- What is move constructor?
- Why should base destructors be virtual?
- What is a pure virtual function?
- What are vtable and vptr?
- Can constructors be virtual?
- What is object slicing?

Common Comparisons

Comparison	One-Line Difference
Constructor vs Destructor	Constructor initializes; destructor cleans up.
Copy Constructor vs Assignment	Copy constructor creates; assignment replaces existing state.
Copy vs Move	Copy duplicates; move transfers ownership.
Virtual vs Pure Virtual	Virtual may have default behavior; pure virtual forces derived implementation.
vtable vs vptr	vtable stores function addresses; vptr points to vtable.
Rule of Three vs Rule of Five	Rule of Five adds move operations.

One-Line Interview Answer

In C++, constructors and destructors manage object lifetime, copy and move operations control resource ownership, and virtual functions provide runtime polymorphism, usually implemented through a vptr in each polymorphic object pointing to a class-level vtable.